
MLP-Mixer: 一种完全基于 MLP 的视觉架构

Ilya Tolstikhin*, Neil Houlsby*, Alexander Kolesnikov*, Lucas Beyer*,
Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner,
Daniel Keysers, Jakob Uszkoreit, Mario Lucic, Alexey Dosovitskiy

* 同等贡献

Google Research, Brain Team

{tolstikhin, neilhoulby, akolesnikov, lbeyer,
xzhai, unterthiner, jessicayung[†], andstein,
keyzers, usz, lucic, adosovitskiy}@google.com

[†] 相关工作完成于 Google AI Residency 期间

Abstract

卷积神经网络 (CNN) 一直是计算机视觉的首选模型。近年来, Vision Transformer 等基于注意力的网络也日益流行。本文表明, 卷积和注意力虽都足以取得良好性能, 却都并非不可或缺。我们提出 *MLP-Mixer*, 一种完全基于多层感知机 (MLP) 的架构。MLP-Mixer 包含两类层: 一类将 MLP 独立应用于各图像块 (即“混合”各位置的特征), 另一类跨图像块应用 MLP (即“混合”空间信息)。在大规模数据集上训练或采用现代正则化方案时, MLP-Mixer 在图像分类基准上取得了有竞争力的成绩, 其预训练和推理成本与最先进模型相当。我们希望这些结果能推动研究超越成熟 CNN 和 Transformer 的既有范畴。¹

1 引言

计算机视觉的发展史表明, 更大数据集与更强计算能力的结合往往会引发范式转变。卷积神经网络 (CNN) 虽长期是计算机视觉的事实标准, 但最近基于自注意力层的替代方案 Vision Transformer [14] (ViT) 取得了最先进性能。ViT 延续了从模型中移除人工设计视觉特征和归纳偏置的长期趋势, 进一步依赖从原始数据中学习。

我们提出 *MLP-Mixer* 架构 (简称“Mixer”): 一种具有竞争力、但概念和技术都很简单的替代方案, 既不使用卷积也不使用自注意力。Mixer 完全基于多层感知机 (MLP), 在空间位置或特征通道维度上反复应用。Mixer 只依赖基本矩阵乘法、数据布局变换 (重塑与转置) 以及标量非线性。

图 1 展示了 Mixer 的宏观结构。其输入是线性投影后的图像块 (亦称词元) 序列, 形状为“图像块 $\times$ 通道”的表格, 并始终保持该维度。Mixer 使用两类 MLP 层: 通道混合 MLP 与词元混合 MLP。通道混合 MLP 使不同通道之间能够通信; 它独立处理每个词元, 以表格的各行作为输入。词元混合 MLP 使不同空间位置 (词元) 之间能够通信; 它独立处理每个通道, 以表格的各列作为输入。两类层交替排列, 使输入的两个维度均可交互。

在极端意义下, 我们的架构可视为一种非常特殊的 CNN: 用 1×1 卷积进行通道混合, 并用具有完整感受野且共享参数的单通道深度卷积进行词元混合。但反之不成立, 因为典型 CNN 并非 Mixer 的特例。此外, 卷积比 MLP 中的普通矩阵乘法更复杂, 因为它需要额外且代价高昂的矩阵乘法归约和/或专门实现。

¹MLP-Mixer 的代码将发布于 https://github.com/google-research/vision_transformer

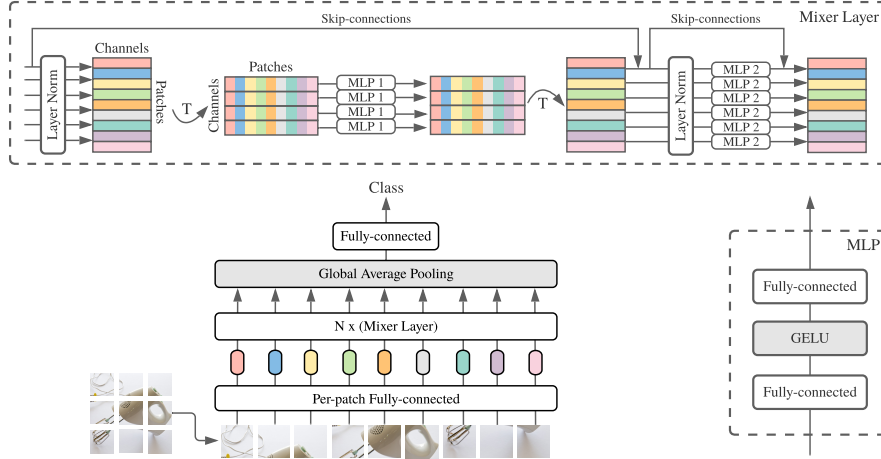


Figure 1: MLP-Mixer 由逐图像块线性嵌入、Mixer 层和分类器头组成。每个 Mixer 层包含一个词元混合 MLP 和一个通道混合 MLP，二者均由两个全连接层及 GELU 非线性构成。其他组件包括跳跃连接、dropout 和通道上的层归一化。

尽管结构简单，Mixer 仍取得了有竞争力的结果。在大规模数据集（即约 $\sim 100M$ 张图像）上预训练时，其准确率/成本权衡接近此前由 CNN 和 Transformer 达到的最先进水平。其中包括在 ILSVRC2012 “ImageNet” [13] 上 87.94% 的 top-1 验证准确率。在规模较适中的数据（即 $\sim 1-10M$ 张图像）上预训练并结合现代正则化技术 [49, 54] 时，Mixer 同样表现强劲。不过，与 ViT 类似，它仍略逊于专门设计的 CNN 架构。

2 Mixer 架构

现代深度视觉架构由如下特征混合层构成：(i) 在给定空间位置内混合，(ii) 在不同空间位置间混合，或同时进行二者。在 CNN 中，(ii) 通过 $N \times N$ 卷积 ($N > 1$) 和池化实现。更深层神经元具有更大的感受野 [1, 28]。同时， 1×1 卷积执行 (i)，更大的卷积核则同时执行 (i) 和 (ii)。在 Vision Transformer 及其他基于注意力的架构中，自注意力层同时实现 (i) 和 (ii)，MLP 块则执行 (i)。Mixer 架构的核心思想，是明确分离逐位置的通道混合操作 (i) 与跨位置的词元混合操作 (ii)。两类操作都由 MLP 实现。图 1 总结了该架构。

Mixer 的输入是由 S 个互不重叠图像块组成的序列，每个图像块均投影到所需的隐藏维度 C 。由此得到二维实值输入表 $\mathbf{X} \in \mathbb{R}^{S \times C}$ 。若原始输入图像分辨率为 (H, W) ，每个图像块分辨率为 (P, P) ，则图像块数量为 $S = HW/P^2$ 。所有图像块均使用同一个投影矩阵进行线性投影。Mixer 由多个尺寸相同的层组成，每层包含两个 MLP 块。第一个是词元混合 MLP：它作用于 $\mathbf{X}$ 的列（即应用于转置后的输入表 $\mathbf{X}^T$ ），映射 $\mathbb{R}^S \mapsto \mathbb{R}^S$ ，并在所有列间共享。第二个是通道混合 MLP：它作用于 $\mathbf{X}$ 的行，映射 $\mathbb{R}^C \mapsto \mathbb{R}^C$ ，并在所有行间共享。每个 MLP 块包含两个全连接层，以及独立应用于输入数据张量每一行的非线性。省略层索引后，Mixer 层可写为：

$$\begin{aligned} \mathbf{U}_{*,i} &= \mathbf{X}_{*,i} + \mathbf{W}_2 \sigma(\mathbf{W}_1 \text{LayerNorm}(\mathbf{X})_{*,i}), \quad \text{for } i = 1 \dots C, \\ \mathbf{Y}_{j,*} &= \mathbf{U}_{j,*} + \mathbf{W}_4 \sigma(\mathbf{W}_3 \text{LayerNorm}(\mathbf{U})_{j,*}), \quad \text{for } j = 1 \dots S. \end{aligned} \quad (1)$$

其中 σ 为逐元素非线性（GELU [16]）。 D_S 与 D_C 分别是词元混合和通道混合 MLP 中可调的隐藏宽度。注意 D_S 的选择与输入图像块数量无关。因此，网络计算复杂度随输入图像块数量线性增长，不同于复杂度为二次的 ViT。由于 D_C 与图像块大小无关，总复杂度与图像像素数呈线性关系，与典型 CNN 相同。

如上所述，同一个通道混合 MLP（词元混合 MLP）被应用于 $\mathbf{X}$ 的每一行（列）。在每层内绑定通道混合 MLP 的参数是自然的选择，因为它提供了卷积的一项突出性质——位置不变性。然而，跨通道绑定参数远不常见。例如，某些 CNN 使用的可分离卷积 [9, 40] 会独立地对每个通道应用卷积。不过，可分离卷积为每个通道使用不同卷积核，而 Mixer 中的词元混合 MLP 在所有通道间共享同一个（具有完整感受野的）核。参数绑定可防止架构在增大隐藏维度 C 或序列长度 S 时增长过快，并显著节省内存。令人意外的是，这一选择并不影响实证性能，见补充材料 A.1。

Mixer 中每一层（初始图像块投影层除外）的输入尺寸都相同。这种“各向同性”设计最接近同样采用固定宽度的 Transformer，或其他领域的深层 RNN。它不同于多数具有金字塔结构的 CNN：更深层的输入分辨率更低，但通道更多。尽管这些是典型设计，也存在其他组合，如各向同性 ResNet [38] 和金字塔式 ViT [52]。

除 MLP 层外，Mixer 还使用其他标准架构组件：跳跃连接 [15] 与层归一化 [2]。不同于 ViT，Mixer 不使用位置嵌入，因为词元混合 MLP 对输入词元顺序敏感。最后，Mixer 使用标准分类头，即全局平均池化层后接线性分类器。整个架构可用 JAX/Flax 简洁实现，代码见补充材料 E。

3 实验

我们在一系列中小规模下游分类任务上，评估使用中到大规模数据集预训练的 MLP-Mixer 模型。我们关注三个主要指标：(1) 下游任务准确率；(2) 预训练的总计算成本，这对在上游数据集上从头训练模型十分重要；(3) 测试时吞吐量，这是实践者关心的指标。我们的目标并非展示最先进结果，而是说明：一个简单的 MLP 模型竟能与当今最优秀的卷积和注意力模型竞争。

下游任务 我们使用常见下游任务，包括采用原始验证标签 [13] 和清理后 ReaL 标签 [5] 的 ILSVRC2012 “ImageNet” (1.3M 个训练样本, 1k 类)、CIFAR-10/100 (50k 个样本, 10/100 类) [23]、Oxford-IIIT Pets (3.7k 个样本, 36 类) [32] 和 Oxford Flowers-102 (2k 个样本, 102 类) [31]。我们还使用视觉任务适应基准 VTAB-1k，其中包含 19 个多样化数据集，每个有 1k 个训练样本 [58]。

预训练 我们遵循标准迁移学习设置：先预训练，再在下游任务上微调。模型在两个公开数据集上预训练：ILSVRC2021 ImageNet，以及包含 21k 类、14M 张图像的 ILSVRC2012 超集 ImageNet-21k [13]。为评估更大规模下的性能，我们还在专有数据集 JFT-300M 上训练，该数据集含 300M 个样本和 18k 类 [44]。按照 Dosovitskiy et al. [14], Kolesnikov et al. [22]，我们相对于下游任务测试集对所有预训练数据集去重。所有模型均以 224 分辨率预训练，使用 Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)、10k 步线性学习率预热及线性衰减、4096 的批量大小、权重衰减，以及全局范数 1 的梯度裁剪。对 JFT-300M，除随机水平翻转外还采用 Szegedy et al. [45] 的裁剪技术预处理图像。对 ImageNet 和 ImageNet-21k，我们采用额外的数据增强与正则化技术，具体包括 RandAugment [12]、mixup [60]、dropout [43] 和随机深度 [19]。这组技术受 *timm* 库 [54] 和 Touvron et al. [48] 启发。超参数详情见补充材料 B。

微调 我们使用带动量的 SGD、批量大小 512、全局范数 1 的梯度裁剪，以及带线性预热的余弦学习率调度进行微调。微调时不使用权重衰减。遵循惯例 [22, 48]，微调分辨率也高于预训练分辨率。由于图像块分辨率固定，这会增加输入块数量（设从 S 增至 S' ），因此需调整 Mixer 词元混合 MLP 块的形状。形式上，式 (1) 的输入左乘权重矩阵 $\mathbf{W}_1 \in \mathbb{R}^{D_S \times S}$ ，改变输入维度 S 时必须调整该操作。为此，我们按图像块数量的比例将隐藏层宽度从 D_S 增至 $D_{S'}$ ，并用对角线上含有 $\mathbf{W}_2$ 副本的块对角矩阵初始化扩大的权重矩阵 $\mathbf{W}'_2 \in \mathbb{R}^{D_{S'} \times S'}$ 。该方案仅允许 $S' = K^2 S$ ，其中 $K \in \mathbb{N}$ 。详情见补充材料 C。在 VTAB-1k 基准上，我们遵循 BiT-HyperRule [22]，分别对输入图像较小和较大的数据集以 224 和 448 分辨率微调 Mixer 模型。

指标 我们评估模型计算成本与质量之间的权衡。前者采用两个指标：(1) TPU-v3 加速器上的总预训练时间，综合每种训练设置的理论 FLOPs、相关硬件上的计算效率和数据效率；(2) TPU-v3 上以图像/秒/核心计的吞吐量。由于不同规模模型可能适合不同批量大小，我们扫描批量大小并报告各模型的最高吞吐量。模型质量主要考察微调后的下游 top-1 准确率。在两处微调所有模型代价过高的场景（图 3 右侧和图 4），我们报告通过求解冻结图像表征与标签之间的 ℓ_2 正则化线性回归问题而得到的少样本准确率。

模型 我们将表 1 汇总的多种 Mixer 配置与最新的最先进 CNN 和注意力模型比较。在所有图表中，基于 MLP 的 Mixer 模型以粉色 (●) 标记，卷积模型以黄色 (●) 标记，注意力模型以蓝色 (●) 标记。Vision Transformer (ViT) 的模型规模和图像块分辨率与 Mixer 相近。HaloNet 是采用类 ResNet 结构的注意力模型，以局部自注意力层替代 3×3 卷积 [51]。我们重点考察高效的“HaloNet-H4 (base 128, Conv-12)”模型，它是更宽 HaloNet-H4 的混合变体，其中部分自注意力层被卷积替代；含注意力和卷积的 HaloNet 仍标为蓝色 (●)。Big Transfer (BiT) [22] 是针对迁移学习优化的 ResNet。NFNet [7] 是为 ImageNet 分类进行多项优化、无需归一化的 ResNet，我们采用 NFNet-F4+。对 EfficientNet 架构，我们考虑 MPL [34] 和 ALIGN [21]。MPL 在 JFT-300M 图像上进行超大规模预训练，使用来自

Table 1: Mixer 架构的规格。“B”“L”和“H”(base、large、huge) 模型规模沿用 Dosovitskiy et al. [14]。简写“B/16”表示采用 16×16 分辨率图像块的 base 规模模型。参数量按输入分辨率 224 报告，不含分类器头权重。

规格	S/32	S/16	B/32	B/16	L/32	L/16	H/14
层数	8	8	12	12	24	24	32
图像块分辨率 $P \times P$	32×32	16×16	32×32	16×16	32×32	16×16	14×14
隐藏维度 C	512	512	768	768	1024	1024	1280
序列长度 S	49	196	49	196	49	196	256
MLP 维度 D_C	2048	2048	3072	3072	4096	4096	5120
MLP 维度 D_S	256	256	384	384	512	512	640
参数量 (M)	19	18	60	59	206	207	431

Table 2: 迁移性能、推理吞吐量与训练成本。各行按推理吞吐量（第五列）排序。在相近成本下，Mixer 的迁移准确率与最先进模型相当。Mixer 模型以 448 分辨率微调；性能为三次微调的平均值，标准差小于 0.1。

	ImNet top-1	ReaL top-1	5 项平均 top-1	VTAB-1k 19 项任务	吞吐量 图像/秒/核心	TPUv3 核心天
在 ImageNet-21k (公开) 上预训练						
● HaloNet [51]	85.8	—	—	—	120	0.10k
● Mixer-L/16	84.15	87.86	93.91	74.95	105	0.41k
● ViT-L/16 [14]	85.30	88.62	94.39	72.72	32	0.18k
● BiT-R152x4 [22]	85.39	—	94.04	70.64	26	0.94k
在 JFT-300M (专有) 上预训练						
● NFNet-F4+ [7]	89.2	—	—	—	46	1.86k
● Mixer-H/14	87.94	90.18	95.71	75.33	40	1.01k
● BiT-R152x4 [22]	87.54	90.54	95.33	76.29	26	9.90k
● ViT-H/14 [14]	88.55	90.72	95.97	77.63	15	2.30k
在无标签或弱标签数据 (专有) 上预训练						
● MPL [34]	90.0	91.12	—	—	—	20.48k
● ALIGN [21]	88.64	—	—	79.99	15	14.82k

ImageNet 的元伪标签而非原始标签，我们与 EfficientNet-B6-Wide 比较。ALIGN 以对比方式在噪声网络图文对上预训练图像和语言编码器，我们与其最佳 EfficientNet-L2 图像编码器比较。

3.1 主要结果

表 2 将最大的 Mixer 模型与文献中的最先进模型进行比较。“ImNet”和“ReaL”分别指原始 ImageNet 验证标签 [13] 与清理后的 ReaL 标签 [5]。“Avg. 5”表示五个下游任务 (ImageNet、CIFAR-10、CIFAR-100、Pets、Flowers) 的平均性能。图 2 (左) 展示准确率-计算量前沿。在 ImageNet-21k 上采用额外正则化预训练时，Mixer 整体表现强劲 (ImageNet top-1 为 84.15%)，但略逊于其他模型²。此时正则化必不可少，否则 Mixer 会过拟合，这与 ViT 的观察一致 [14]。在 ImageNet 上随机初始化训练 Mixer 时结论也相同 (见第 3.2 节)：Mixer-B/16 在分辨率 224 下达到尚可的 76.4%，但容易过拟合。该成绩接近普通 ResNet50，却落后于 ImageNet “从头训练”设置中的最先进 CNN/混合模型，如 84.7% 的 BotNet [42] 和 86.5% 的 NFNet [7]。

随着上游数据集增大，Mixer 性能显著提升。尤其是 Mixer-H/14 在 ImageNet 上达到 87.94% top-1，比 BiT-ResNet152x4 高 0.5%，仅比 ViT-H/14 低 0.5%。值得注意的是，它比 ViT-H/14 快 2.5 倍，几乎比 BiT 快一倍。总体而言，图 2 (左) 支持我们的主要主张：在准确

²表 2 对每个预训练数据集选取各类中准确率最高的模型，它们均采用较大分辨率 (448 及以上)。然而，以较小分辨率微调通常只损失少量准确率，却能显著提升测试吞吐量。例如，在 ImageNet-21k 上预训练后，以 224 分辨率微调的 Mixer-L/16 达到 82.84% ImageNet top-1，吞吐量为 420 图像/秒/核心；以 384 分辨率微调的 ViT-L/16 达到 85.15%，吞吐量为 80 图像/秒/核心 [14]；以 384 分辨率微调的 HaloNet 达到 85.5%，吞吐量为 258 图像/秒/核心 [51]。

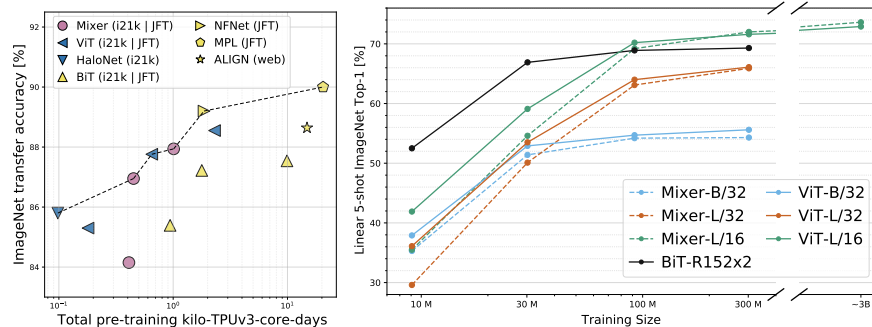


Figure 2: 左：表 2 中最先进模型的 ImageNet 准确率/训练成本 Pareto 前沿（虚线）。模型在 ImageNet-21k、JFT（有标签，MPL 使用伪标签）或网络图文对上预训练。Mixer 与这些高性能 ResNet、ViT 和混合模型相当，并与 HaloNet、ViT、NFNet 和 MPL 同处前沿。右：随着数据规模增大，Mixer（实线）追上或超过 BiT（点线）和 ViT（虚线）。曲线上各点使用相同预训练计算量，分别对应在 JFT-300M 的 3%、10%、30% 和 100% 上训练 233、70、23 和 7 个 epoch。~3B 处附加点表示以相同步数在更大的 JFT-3B 上预训练。Mixer 随数据增长的提升快于 ResNet，甚至快于 ViT；大型 Mixer 与 ViT 的差距随之缩小。

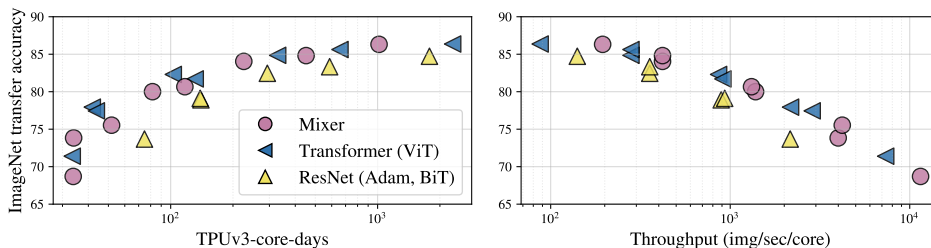


Figure 3: 模型规模的作用。不同规模 ViT、BiT 和 Mixer 的 ImageNet 验证 top-1 准确率与总预训练计算量（左）及吞吐量（右）的关系。所有模型均在 JFT-300M 上预训练并以 224 分辨率微调，该分辨率低于图 2（左）。

率-计算量权衡上，Mixer 可与传统神经网络架构竞争；该图还显示，即使跨架构类别，总预训练成本与下游准确率也明显相关。

表中的 BiT-ResNet152x4 使用带动量 SGD 和长调度预训练。由于 Adam 通常收敛更快，图 2（左）还纳入了 Dosovitskiy et al. [14] 在 JFT-300M 上用 Adam 预训练的 BiT-R200x3。该 ResNet 准确率稍低，但预训练计算量显著更少。图中还报告了较小的 ViT-L/16 和 Mixer-L/16。

3.2 模型规模的作用

上一节聚焦计算规模高端的（大型）模型，下面转向较小的 Mixer。模型可沿两个独立方向扩展：(1) 预训练时增大模型（层数、隐藏维度、MLP 宽度）；(2) 微调时提高输入图像分辨率。前者同时影响预训练计算量和测试吞吐量，后者仅影响吞吐量。除非另有说明，均以 224 分辨率微调。

我们将多种 Mixer 配置（见表 1）与相近规模的 ViT 及用 Adam 预训练的 BiT 比较，结果见表 3 和图 3。在 ImageNet 上从头训练时，Mixer-B/16 达到合理的 76.44% top-1，比 ViT-B/16 低 3%。训练曲线（未报告）显示两者训练损失非常接近，即 Mixer-B/16 过拟合更严重；在 Mixer-L/16 与 ViT-L/16 上差异更明显。随着预训练数据集增长，Mixer 性能稳定提高。在 JFT-300M 上预训练并以 224 分辨率微调的 Mixer-H/14 在 ImageNet 上仅落后 ViT-H/14 0.3%，却快 2.2 倍。图 3 清楚表明：Mixer 在小规模端略低于前沿，但在大规模端稳居前沿。

3.3 预训练数据集规模的作用

上述结果表明，在更大数据集上预训练能显著提升 Mixer 性能。下面更细致地研究这一效应。

Table 3: 不同模型规模和预训练数据集规模下, Mixer 与文献中其他模型的性能。“Avg. 5”表示五个下游任务的平均性能。Mixer 和 ViT 取三次微调平均值, 标准差小于 0.15。(‡) 由相同模型在 JFT-300M 上不加额外正则化的报告值外推。(Ⓐ) 由 Dosovitskiy et al. [14] 的作者私下提供。各行按吞吐量排序。

	图像 尺寸	预训练 Epoch	ImNet top-1	ReaL top-1	5 项平均 top-1	吞吐量 (图像/秒/核心)	TPUv3 核心天
在 ImageNet 上预训练 (额外正则化)							
● Mixer-B/16	224	300	76.44	82.36	88.33	1384	0.01k ^(‡)
● ViT-B/16 (Ⓐ)	224	300	79.67	84.97	90.79	861	0.02k ^(‡)
● Mixer-L/16	224	300	71.76	77.08	87.25	419	0.04k ^(‡)
● ViT-L/16 (Ⓐ)	224	300	76.11	80.93	89.66	280	0.05k ^(‡)
在 ImageNet-21k 上预训练 (额外正则化)							
● Mixer-B/16	224	300	80.64	85.80	92.50	1384	0.15k ^(‡)
● ViT-B/16 (Ⓐ)	224	300	84.59	88.93	94.16	861	0.18k ^(‡)
● Mixer-L/16	224	300	82.89	87.54	93.63	419	0.41k ^(‡)
● ViT-L/16 (Ⓐ)	224	300	84.46	88.35	94.49	280	0.55k ^(‡)
● Mixer-L/16	448	300	83.91	87.75	93.86	105	0.41k ^(‡)
在 JFT-300M 上预训练							
● Mixer-S/32	224	5	68.70	75.83	87.13	11489	0.01k
● Mixer-B/32	224	7	75.53	81.94	90.99	4208	0.05k
● Mixer-S/16	224	5	73.83	80.60	89.50	3994	0.03k
● BiT-R50x1	224	7	73.69	81.92	—	2159	0.08k
● Mixer-B/16	224	7	80.00	85.56	92.60	1384	0.08k
● Mixer-L/32	224	7	80.67	85.62	93.24	1314	0.12k
● BiT-R152x1	224	7	79.12	86.12	—	932	0.14k
● BiT-R50x2	224	7	78.92	86.06	—	890	0.14k
● BiT-R152x2	224	14	83.34	88.90	—	356	0.58k
● Mixer-L/16	224	7	84.05	88.14	94.51	419	0.23k
● Mixer-L/16	224	14	84.82	88.48	94.77	419	0.45k
● ViT-L/16	224	14	85.63	89.16	95.21	280	0.65k
● Mixer-H/14	224	14	86.32	89.14	95.49	194	1.01k
● BiT-R200x3	224	14	84.73	89.58	—	141	1.78k
● Mixer-L/16	448	14	86.78	89.72	95.13	105	0.45k
● ViT-H/14	224	14	86.65	89.56	95.57	87	2.30k
● ViT-L/16 [14]	512	14	87.76	90.54	95.63	32	0.65k

为研究 Mixer 利用不断增加训练样本的能力, 我们在 JFT-300M 的随机子集 (含全部样本的 3%、10%、30% 和 100%) 上分别预训练 Mixer-B/32、Mixer-L/32 和 Mixer-L/16 233、70、23 和 7 个 epoch, 使各模型总步数相同。我们还以相同步数在更大的 JFT-3B [59] (约 3B 张图像、30k 类) 上预训练 Mixer-L/16; 虽不能严格比较, 却可进一步外推规模效应。我们以 ImageNet 线性 5-shot top-1 准确率代理迁移质量, 每次预训练都按最佳上游验证性能早停。结果见图 2 (右), 其中也包括 ViT-B/32、ViT-L/32、ViT-L/16 和 BiT-R152x2。

在 JFT-300M 最小子集上预训练时, 所有 Mixer 均严重过拟合。BiT 也过拟合但程度较轻, 可能源于卷积的强归纳偏置。数据集增大时, Mixer-L/32 和 Mixer-L/16 的提升均快于 BiT; Mixer-L/16 持续改善, 而 BiT 趋于平台。

ViT 也有相同结论, 与 Dosovitskiy et al. [14] 一致, 但大型 Mixer 的相对提升更加明显。Mixer-L/16 与 ViT-L/16 的性能差距随数据规模缩小, 似乎 Mixer 从数据增长中获益甚至超过 ViT。一种推测仍是归纳偏置差异: ViT 自注意力层使所学函数具有某些性质, 与真实底层分布的相容性更低, 不及 Mixer 架构发现的性质。

3.4 对输入置换的不变性

本节研究 Mixer 与 CNN 架构归纳偏置的差异。具体而言, 我们遵循第 3 节的预训练设置, 在 JFT-300M 上训练 Mixer-B/16 和 ResNet50x1, 并采用两种输入变换之一: (1) 打乱 16×16 图像块的顺序, 并用共享置换打乱每个块内的像素; (2) 在整幅图像中全局置换像素。所有图像使用同一置换。图 4 (下) 报告模型在 ImageNet 上的线性 5-shot top-1 准确率, 图 4 (上) 展示部分原图及两种变换版本。符合预期, Mixer 对图像块顺序和块内像素顺序不变 (蓝绿曲线完全重合)。而 ResNet 的强归纳偏置依赖图像内特定像素顺序, 置换图像块后性能显著下降。值得注意的是, 全局置换像素时, Mixer 的性能下降 (~45%) 远小于 ResNet (~75%)。

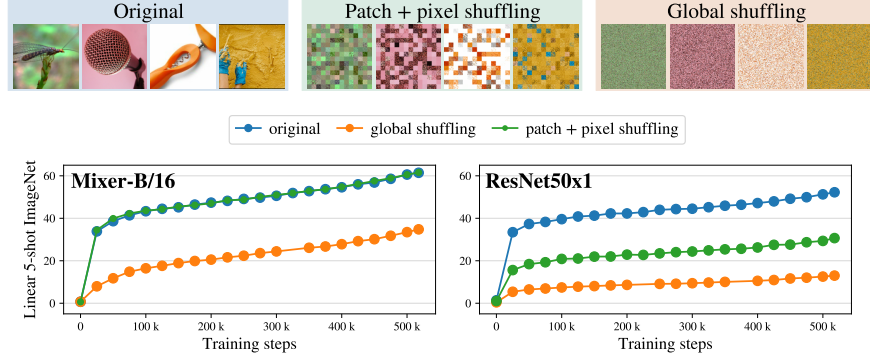


Figure 4: 上: ImageNet 输入样例: 置换内容前 (左); 打乱 16×16 图像块及块内像素后 (中); 全局打乱像素后 (右)。下: 用三种对应输入流水线训练的 Mixer-B/16 (左) 和 ResNet50x1 (右)。

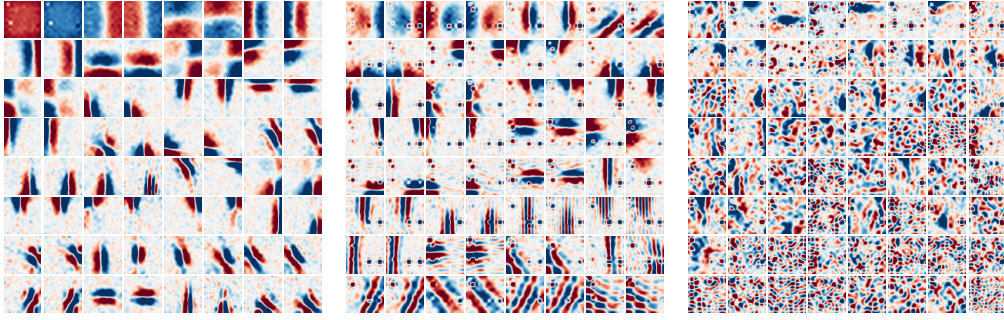


Figure 5: 在 JFT-300M 上训练的 Mixer-B/16 中, 第一 (左)、第二 (中) 和第三 (右) 个词元混合 MLP 的隐藏单元。每个单元有 196 个权重, 分别对应 14×14 个输入图像块。我们将单元配对, 以突出相位相反的核的出现, 并按滤波器频率排序。卷积滤波器核的每个权重对应输入图像的一个像素; 与之不同, 左列任一图中的一个权重对应输入图像的一个特定 16×16 图像块。完整图见补充材料 D。

3.5 可视化

通常可观察到 CNN 的最初几层倾向学习作用于图像局部像素的类 Gabor 检测器。相比之下, Mixer 的词元混合 MLP 允许全局信息交换, 因此自然要问它是否以类似方式处理信息。图 5 展示了在 JFT-300M 上训练的 Mixer 前三个词元混合 MLP 的隐藏单元。词元混合 MLP 可在不同空间位置间进行全局通信; 部分所学特征作用于整幅图像, 另一些作用于较小区域。更深层似乎没有可明确辨认的结构。与 CNN 类似, 我们观察到许多相位相反的特征检测器对 [39]。所学单元结构取决于超参数。第一嵌入层的图见补充材料 D 中的图 7。

4 相关工作

MLP-Mixer 是一种新的计算机视觉架构, 不使用卷积层或自注意力层, 因而不同于以往成功架构; 不过, 其设计选择可追溯至 CNN [24, 25] 与 Transformer [50] 文献中的思想。

自 AlexNet [24] 超越基于人工图像特征的主流方法 [35] 后, CNN 一直是计算机视觉的事实标准。许多工作致力于改进 CNN: Simonyan and Zisserman [41] 表明仅用小型 3×3 卷积核即可训练最先进模型; He et al. [15] 将跳跃连接与批归一化 [20] 结合, 使极深网络训练成为可能并进一步提升性能。另一条重要路线研究分组卷积 [57]、深度卷积 [9, 17] 等稀疏卷积。与词元混合 MLP 精神相似, Wu et al. [55] 在自然语言处理的深度卷积中共享参数。Hu et al. [18] 和 Wang et al. [53] 用非局部操作增强卷积网络, 部分缓解 CNN 局部处理的限制。Mixer 将小核卷积推至极端: 把核缩小至 1×1 , 使卷积变成独立应用于各空间位置的标准稠密矩阵乘法 (通道混合 MLP)。这本身无法聚合空间信息, 因此我们以跨所有空间位置作用于每个特征的稠密矩阵乘法 (词元混合 MLP) 补偿。Mixer 在“图像块 $\times$ 特征”输入表上按行或按列应用矩阵乘法, 与稀疏卷积工作也密切相关。Mixer 使用跳跃连接 [15] 和归一化层 [2, 20]。

在计算机视觉中，基于自注意力的 Transformer 最初用于生成建模 [8, 33]，之后才在结合类卷积局部性偏置 [37] 或低分辨率图像 [10] 上展现图像识别价值。Dosovitskiy et al. [14] 提出 ViT：局部性偏置更少、却能良好扩展至大数据的纯 Transformer。ViT 在常见视觉基准上取得最先进性能，同时保留 CNN 的鲁棒性 [6]。Touvron et al. [49] 通过广泛正则化在较小数据集上有效训练 ViT。Mixer 借鉴近期 Transformer 架构：其 MLP 块设计源自 Vaswani et al. [50]，将图像转为图像块序列并直接处理其嵌入则源自 Dosovitskiy et al. [14]。

近期许多工作致力于设计更有效的视觉架构。Srinivas et al. [42] 用自注意力层替换 ResNet 的 3×3 卷积；Ramachandran et al. [37]、Tay et al. [47]、Li et al. [26] 和 Bello [3] 设计了采用新型类注意力机制的网络。Mixer 可视作为正交方向上的一步，不依赖局部性偏置和注意力机制。

Lin et al. [27] 的工作密切相关：它利用全连接网络、强数据增强和自编码器预训练，在 CIFAR-10 上取得合理性能。Neyshabur [30] 设计定制正则化与优化算法训练全连接网络，在小规模任务上表现出色。我们则依赖词元和通道混合 MLP，使用标准正则化和优化技术，并有效扩展至大数据。

传统上，在 ImageNet [13] 上评估的网络从随机初始化开始，并采用 Inception 式预处理 [46]；对较小数据集，迁移 ImageNet 模型很常见。但现代最先进模型通常使用更大数据集上的预训练权重，或较新的数据增强与训练策略。例如，Dosovitskiy et al. [14]，Kolesnikov et al. [22]，Mahajan et al. [29]，Pham et al. [34]，Xie et al. [56] 均以大规模预训练推进图像分类前沿。增强或正则化改进的例子包括：Cubuk et al. [11] 通过学习得到的数据增强取得优秀分类性能；Bello et al. [4] 表明，若采用近期训练和增强策略，经典 ResNet 仍接近最先进水平。

5 结论

我们描述了一种非常简单的视觉架构。实验表明，在准确率与训练、推理所需计算资源的权衡上，它不逊于现有最先进方法。我们相信这些结果提出了许多新问题。实践上，可研究模型学到的特征，并辨析其与 CNN、Transformer 所学特征的主要差异（若有）。理论上，我们希望理解这些特征中隐藏的归纳偏置及其最终对泛化的作用。更重要的是，我们希望这些结果推动研究超越基于卷积和自注意力的成熟模型范畴。尤其值得探究的是，这种设计能否适用于 NLP 或其他领域。

Acknowledgements

本工作由柏林和 Zürich 的 Brain 团队完成。感谢 Josip Djolonga 对论文初稿的反馈；感谢 Preetum Nakkiran 提议在像素打乱的输入图像上训练 MLP-Mixer；感谢 Olivier Bousquet、Yann Dauphin 和 Dirk Weissenborn 的有益讨论。

参考文献

- [1] A. Araujo, W. Norris, and J. Sim. Computing receptive fields of convolutional neural networks. *Distill*, 2019. doi: 10.23915/distill.00021. URL <https://distill.pub/2019/computing-receptive-fields>.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [3] I. Bello. LambdaNetworks: Modeling long-range interactions without attention. *arXiv preprint arXiv:2102.08602*, 2021.
- [4] I. Bello, W. Fedus, X. Du, E. D. Cubuk, A. Srinivas, T.-Y. Lin, J. Shlens, and B. Zoph. Revisiting ResNets: Improved training and scaling strategies. *arXiv preprint arXiv:2103.07579*, 2021.
- [5] L. Beyer, O. J. Hénaff, A. Kolesnikov, X. Zhai, and A. van den Oord. Are we done with ImageNet? *arXiv preprint arXiv:2006.07159*, 2020.
- [6] S. Bhojanapalli, A. Chakrabarti, D. Glasner, D. Li, T. Unterthiner, and A. Veit. Understanding robustness of transformers for image classification. *arXiv preprint arXiv:2103.14586*, 2021.

- [7] A. Brock, S. De, S. L. Smith, and K. Simonyan. High-performance large-scale image recognition without normalization. *arXiv preprint arXiv:2102.06171*, 2021.
- [8] R. Child, S. Gray, A. Radford, and I. Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [9] F. Chollet. Xception: Deep learning with depthwise separable convolutions. In *CVPR*, 2017.
- [10] J.-B. Cordonnier, A. Loukas, and M. Jaggi. On the relationship between self-attention and convolutional layers. In *ICLR*, 2020.
- [11] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le. AutoAugment: Learning augmentation policies from data. In *CVPR*, 2019.
- [12] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le. RandAugment: Practical automated data augmentation with a reduced search space. In *CVPR Workshops*, 2020.
- [13] J. Deng, W. Dong, R. Socher, L. Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.
- [14] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.
- [15] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- [16] D. Hendrycks and K. Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.
- [17] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- [18] J. Hu, L. Shen, and G. Sun. Squeeze-and-excitation networks. In *CVPR*, 2018.
- [19] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger. Deep networks with stochastic depth. In *ECCV*, 2016.
- [20] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.
- [21] C. Jia, Y. Yang, Y. Xia, Y.-T. Chen, Z. Parekh, H. Pham, Q. V. Le, Y. Sung, Z. Li, and T. Duerig. Scaling up visual and vision-language representation learning with noisy text supervision. *arXiv preprint arXiv:2102.05918*, 2021.
- [22] A. Kolesnikov, L. Beyer, X. Zhai, J. Puigcerver, J. Yung, S. Gelly, and N. Houlsby. Big transfer (BiT): General visual representation learning. In *ECCV*, 2020.
- [23] A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [24] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *NeurIPS*, 2012.
- [25] Y. LeCun, B. Boser, J. Denker, D. Henderson, R. Howard, W. Hubbard, and L. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1: 541–551, 1989.
- [26] D. Li, J. Hu, C. Wang, X. Li, Q. She, L. Zhu, T. Zhang, and Q. Chen. Involution: Inverting the inference of convolution for visual recognition. *CVPR*, 2021.
- [27] Z. Lin, R. Memisevic, and K. Konda. How far can we go without convolution: Improving fullyconnected networks. In *ICLR, Workshop Track*, 2016.
- [28] W. Luo, Y. Li, R. Urtasun, and R. Zemel. Understanding the effective receptive field in deep convolutional neural networks. In *NeurIPS*, 2016.
- [29] D. Mahajan, R. Girshick, V. Ramanathan, K. He, M. Paluri, Y. Li, A. Bharambe, and L. van der Maaten. Exploring the limits of weakly supervised pretraining. In *ECCV*, 2018.

- [30] B. Neyshabur. Towards learning convolutions from scratch. In *NeurIPS*, 2020.
- [31] M. Nilsback and A. Zisserman. Automated flower classification over a large number of classes. In *ICVGIP*, 2008.
- [32] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar. Cats and dogs. In *CVPR*, 2012.
- [33] N. Parmar, A. Vaswani, J. Uszkoreit, L. Kaiser, N. Shazeer, A. Ku, and D. Tran. Image transformer. In *ICML*, 2018.
- [34] H. Pham, Z. Dai, Q. Xie, M.-T. Luong, and Q. V. Le. Meta pseudo labels. In *CVPR*, 2021.
- [35] A. Pinz. Object categorization. *Foundations and Trends in Computer Graphics and Vision*, 1(4), 2006.
- [36] B. T. Polyak and A. B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.
- [37] P. Ramachandran, N. Parmar, A. Vaswani, I. Bello, A. Levskaya, and J. Shlens. Stand-alone self-attention in vision models. In *NeurIPS*, 2019.
- [38] M. Sandler, J. Baccash, A. Zhmoginov, and Howard. Non-discriminative data or weak model? On the relative importance of data and model resolution. In *ICCV Workshop on Real-World Recognition from Low-Quality Images and Videos*, 2019.
- [39] W. Shang, K. Sohn, D. Almeida, and H. Lee. Understanding and improving convolutional neural networks via concatenated rectified linear units. In *ICML*, 2016.
- [40] L. Sifre. *Rigid-Motion Scattering For Image Classification*. PhD thesis, Ecole Polytechnique, 2014.
- [41] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.
- [42] A. Srinivas, T.-Y. Lin, N. Parmar, J. Shlens, P. Abbeel, and A. Vaswani. Bottleneck transformers for visual recognition. *arXiv preprint arXiv:2101.11605*, 2021.
- [43] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 15(56), 2014.
- [44] C. Sun, A. Shrivastava, S. Singh, and A. Gupta. Revisiting unreasonable effectiveness of data in deep learning era. In *ICCV*, 2017.
- [45] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *CVPR*, 2015.
- [46] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the inception architecture for computer vision. In *CVPR*, 2016.
- [47] Y. Tay, D. Bahri, D. Metzler, D.-C. Juan, Z. Zhao, and C. Zheng. Synthesizer: Rethinking self-attention in transformer models. *arXiv*, 2020.
- [48] H. Touvron, A. Vedaldi, M. Douze, and H. Jegou. Fixing the train-test resolution discrepancy. In *NeurIPS*, 2019.
- [49] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou. Training data-efficient image transformers & distillation through attention. *arXiv preprint arXiv:2012.12877*, 2020.
- [50] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- [51] A. Vaswani, P. Ramachandran, A. Srinivas, N. Parmar, B. Hechtman, and J. Shlens. Scaling local self-attention for parameter efficient visual backbones. *arXiv preprint arXiv:2103.12731*, 2021.
- [52] W. Wang, E. Xie, X. Li, D.-P. Fan, K. Song, D. Liang, T. Lu, P. Luo, and L. Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. *arXiv preprint arXiv:2102.12122*, 2021.
- [53] X. Wang, R. Girshick, A. Gupta, and K. He. Non-local neural networks. In *CVPR*, 2018.

- [54] R. Wightman. Pytorch image models. <https://github.com/rwightman/pytorch-image-models>, 2019.
- [55] F. Wu, A. Fan, A. Baeviski, Y. Dauphin, and M. Auli. Pay less attention with lightweight and dynamic convolutions. In *ICLR*, 2019.
- [56] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le. Self-training with noisy student improves imagenet classification. In *CVPR*, 2020.
- [57] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He. Aggregated residual transformations for deep neural networks. *arXiv preprint arXiv:1611.05431*, 2016.
- [58] X. Zhai, J. Puigcerver, A. Kolesnikov, P. Ruysen, C. Riquelme, M. Lucic, J. Djolonga, A. S. Pinto, M. Neumann, A. Dosovitskiy, et al. A large-scale study of representation learning with the visual task adaptation benchmark. *arXiv preprint arXiv:1910.04867*, 2019.
- [59] X. Zhai, A. Kolesnikov, N. Houlsby, and L. Beyer. Scaling vision transformers. *arXiv preprint arXiv:2106.04560*, 2021.
- [60] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz. mixup: Beyond empirical risk minimization. In *ICLR*, 2018.

A 未能带来帮助の尝试

A.1 修改 token-mixing MLP

我们对若干设想进行了消融实验，试图改进在 JFT-300M 上预训练的不同规模 Mixer 模型中的 token-mixing MLP。

解除参数绑定 (不共享参数) Mixer 层中的 token-mixing MLP 在输入表 $\mathbf{X} \in \mathbb{R}^{S \times C}$ 的各列之间共享。换言之，对 C 个不同特征中的每一个都应用同一个 MLP。另一种做法是引入 C 个权重独立的不同 MLP，这实际上会使参数数量扩大为原来的 C 倍。我们没有观察到任何明显改进。

对通道进行分组 Token-mixing MLP 以 S 维向量作为输入。每个这样的向量都包含单个特征在 S 个不同空间位置上的取值。换言之，token-mixing MLP 每次仅查看一个通道来进行运算。也可以将 $\mathbf{X} \in \mathbb{R}^{S \times C}$ 中相邻的 G 列拼接起来，从而对通道分组，并将其重塑为维度为 $(S \cdot G) \times (C/G)$ 的矩阵。这会将 MLP 的输入维度从 S 增加到 $G \cdot S$ ，并将待处理向量的数量从 C 减少到 C/G 。此时，MLP 在混合 token 时会同时查看多个通道。这种列向量拼接方式使 ImageNet 上线性 5-shot top-1 准确率的提升不足 1-2%。

我们还尝试了另一种版本，将上述简单重塑替换为以下过程：(1) 引入 G 个（具有可训练参数的）线性函数，将 $\mathbb{R}^C$ 投影到 $\mathbb{R}^{C/G}$ 。(2) 使用这些函数，将 $\mathbf{X} \in \mathbb{R}^{S \times C}$ 中的 S 行 (token) 分别映射为 G 个不同的 (C/G) 维向量。由此，每个 token 都会得到 G 个不同的“视图”，每个视图由 C/G 个特征组成。(3) 最后，对于 C/G 个特征中的每一个，拼接对应于 G 个不同视图的向量。这将得到一个维度为 $(S \cdot G) \times (C/G)$ 的矩阵。其思想是，MLP 在混合 token 时可以查看原始通道的 G 个不同视图。对于 Mixer-S/32 架构，该版本将 ImageNet top-5 准确率提高了 3-4%，但在更大规模的模型上没有带来任何改进。

金字塔 Mixer 中的所有层均保持相同的各向同性设计。近期对 ViT 架构的改进表明，这种设计可能并非最优 [52]。我们尝试使用 token-mixing MLP，将 S 个输入 token 映射为 $S' < S$ 个输出 token，以减少 token 数量。初步实验表明，在 JFT-300M 上，此类模型能够在几乎不损失性能的情况下显著缩短训练时间，但我们未能将这些结果迁移到 ImageNet 或 ImageNet-21k。不过，鉴于金字塔是一种流行的设计，在其他视觉任务中探索这一设计仍可能很有前景。

A.2 微调

沿用 BiT [22] 和 ViT [14] 的思路，我们还尝试在微调期间使用 mixup [60] 和 Polyak 平均 [36]。然而，这些方法未能带来一致的改进，因此我们弃用了它们。我们也实验了在微调期间使用 inception cropping [45]，但同样没有取得任何改进。我们在所有规模、经 JFT-300M 预训练的 Mixer 模型上进行了这些实验。

Table 4: 预训练 Mixer 模型所使用的超参数设置。

模型	数据集	轮数	lr	wd	RandAug.	Mixup	Dropout	随机深度
Mixer-B	ImNet	300	0.001	0.1	15	0.5	0.0	0.1
Mixer-L	ImNet	300	0.001	0.1	15	0.5	0.0	0.1
Mixer-B	ImNet-21k	300	0.001	0.1	10	0.2	0.0	0.1
Mixer-L	ImNet-21k	300	0.001	0.1	20	0.5	0.0	0.1
Mixer-S	JFT-300M	5	0.003	0.03	—	—	—	—
Mixer-B	JFT-300M	7	0.003	0.03	—	—	—	—
Mixer-L	JFT-300M	7/14	0.001	0.03	—	—	—	—
Mixer-H	JFT-300M	14	0.001	0.03	—	—	—	—

B 预训练：超参数、数据增强与正则化

表 4 给出了预训练 Mixer 模型时所使用的最优超参数设置。

在 ImageNet 和 ImageNet-21k 上进行预训练时，我们使用了额外的数据增强和正则化。对于 RandAugment [12]，我们始终使用两个增强层，并在集合 $\{0, 10, 15, 20\}$ 中扫描幅度参数 m 。对于 mixup [60]，我们在集合 $\{0.0, 0.2, 0.5, 0.8\}$ 中扫描混合强度 p 。对于 dropout [43]，我们尝试了 0.0 和 0.1 两种丢弃率 d 。对于随机深度，我们遵循原论文 [19]，将丢弃层的概率从 0.0（第一个 MLP）线性增加到 s （最后一个 MLP），其中尝试了 $s \in \{0.0, 0.1\}$ 。最后，我们分别在 $\{0.003, 0.001\}$ 和 $\{0.1, 0.01\}$ 中扫描学习率 lr 与权重衰减 wd 。

C 微调：超参数与更高的图像分辨率

除非另有说明，模型均在分辨率 224 下进行微调。我们遵循 [14] 的设置。仅有以下几点不同：(1) 我们从网格搜索中排除 $lr = 0.001$ ，并为 CIFAR-10、CIFAR-100、Flowers 和 Pets 加入 $lr = 0.06$ 。(2) 对 VTAB-1k，我们在 $lr \in \{0.003, 0.01, 0.03\}$ 上进行网格搜索。(3) 我们在评估期间尝试两种不同的预处理方式：(i) “resize-crop”：首先将图像缩放至 256×256 像素，然后进行大小为 224×224 像素的中心裁剪。(ii) “resmall-crop”：首先将图像的短边缩放至 256 像素，然后进行大小为 224×224 像素的中心裁剪。对于正文表 3 中报告的 Mixer 和 ViT 模型，我们在 ImageNet、Pets、Flowers、CIFAR-10 和 CIFAR-100 上使用了 (ii)。对于正文表 3 中报告的 BiT 模型，我们使用了相同设置，唯一的例外是在 ImageNet 上使用 (i)。对于正文表 2 中报告的 Mixer 模型，我们在全部 5 个下游数据集上都使用了 (i)。

已有研究表明，采用高于预训练时的分辨率进行微调，可以显著提升现有视觉模型的迁移性能 [48, 22, 14]。因此，我们也将这一技术应用于 Mixer。向模型输入更高分辨率的图像时，我们不改变 patch 大小，因而会得到更长的 token 序列。必须调整 token-mixing MLP 才能处理这些更长的序列。我们实验了若干方案，下面介绍其中最成功的一种。

为简单起见，我们假设图像分辨率扩大整数倍 K 。token 序列的长度 S 扩大为原来的 K^2 倍。我们也将 token-mixing MLP 的隐藏宽度 D_S 扩大为原来的 K^2 倍。此时，需要用预训练 MLP 的参数来初始化这个新的（更大的）MLP。为此，我们将输入序列均分为 K^2 个部分，每一部分的长度均为原来的 S ，并对新 MLP 进行初始化，使其使用预训练 MLP 独立、并行地处理所有这些部分。

形式化地说，正式式 1 中原始 MLP 的预训练权重矩阵 $\mathbf{W}_1 \in \mathbb{R}^{D_S \times S}$ 现在将被更大的矩阵 $\mathbf{W}'_1 \in \mathbb{R}^{(K^2 \cdot D_S) \times (K^2 \cdot S)}$ 取代。假设缩放后输入图像的 token 序列由 K^2 个长度均为 S 的 token 序列拼接而成，这些序列通过在空间上将输入划分为 $K \times K$ 个等大部分得到。随后，我们使用一个块对角矩阵初始化 $\mathbf{W}'_1$ ，该矩阵的主对角线上是 $\mathbf{W}_1$ 的副本。MLP 的其他参数采用类似方式处理。

D 权重可视化

为了获得更好的可视化效果，我们根据一种启发式方法对所有隐藏单元进行排序，力求优先展示低频滤波器。对于每个单元，我们还尝试找出与其逆最接近的单元。图 6 中，每个单元

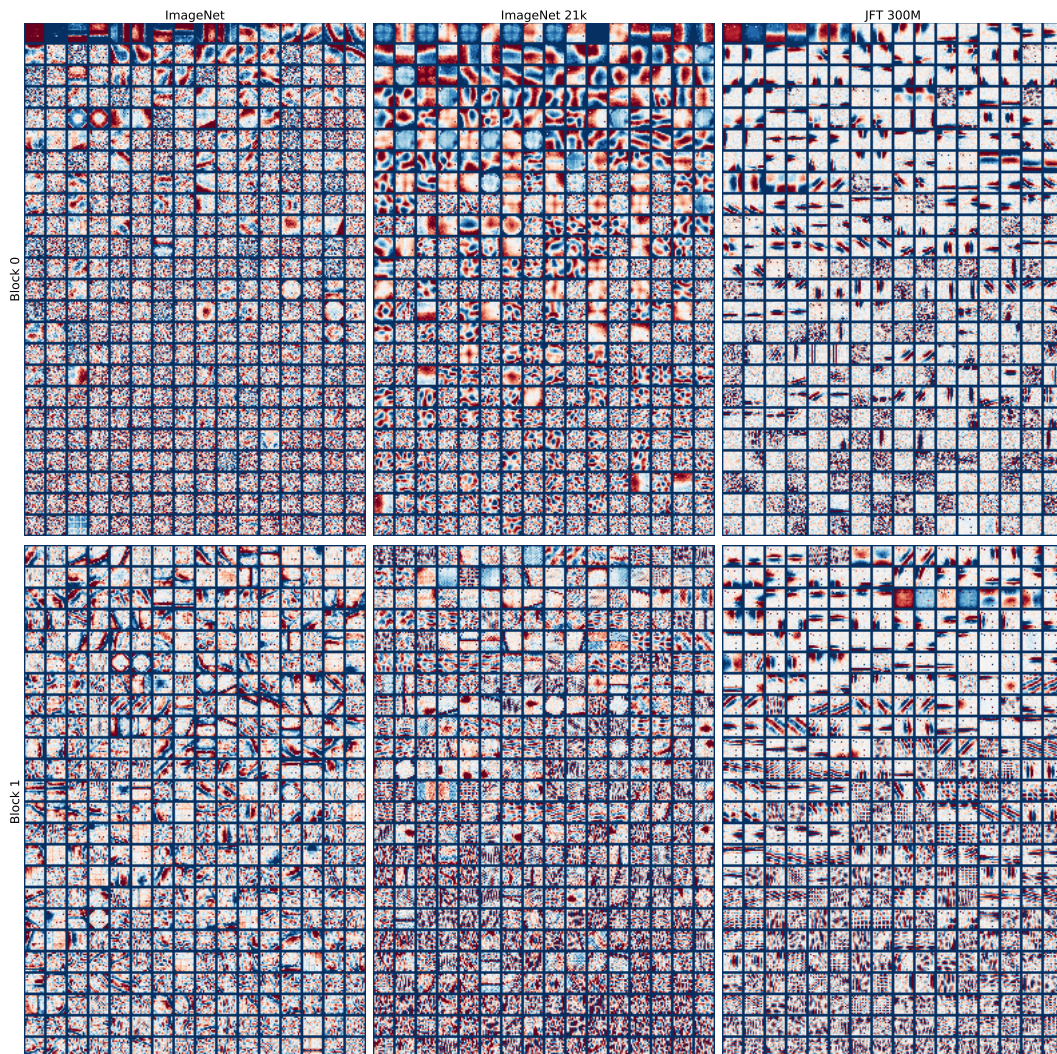


Figure 6: 在三个不同数据集 (列) 上训练的 Mixer-B/16 模型中, 前两个 token-mixing MLP (行) 的所有隐藏全连接单元权重。每个单元有 $14 \times 14 = 196$ 个权重, 与输入 token 的数量相同, 并以一幅 14×14 图像表示。每个块中共有 384 个隐藏单元。

之后都紧跟其最接近的逆。请注意, 在 ImageNet 和 ImageNet-21k 上预训练的模型使用了强数据增强。我们发现, 这会显著影响所学单元的结构。

我们还在图 7 中可视化了不同模型在嵌入层中学到的线性投影单元。有趣的是, 这些单元的性质似乎强烈依赖于模型所使用的 patch 分辨率。在所有 Mixer 模型规模下, 使用分辨率更高的 32×32 patch 都会产生类似 Gabor 的低频线性投影单元, 而在 16×16 分辨率下, 这些单元则没有呈现此类结构。

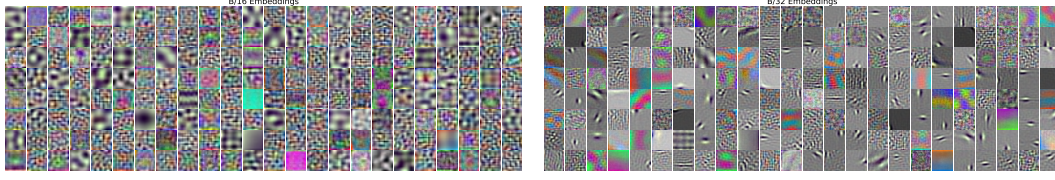


Figure 7: 在 JFT-300M 上预训练的 Mixer-B/16 (左) 与 Mixer-B/32 (右) 模型的嵌入层线性投影单元。使用更高分辨率 32×32 patch 的 Mixer-B/32 模型学到了结构非常鲜明的低频投影单元，而 Mixer-B/16 学到的大多数单元频率较高，且没有清晰结构。

E MLP-Mixer 代码

```

1 import einops
2 import flax.linen as nn
3 import jax.numpy as jnp
4
5 class MlpBlock(nn.Module):
6     mlp_dim: int
7     @nn.compact
8     def __call__(self, x):
9         y = nn.Dense(self.mlp_dim)(x)
10        y = nn.gelu(y)
11        return nn.Dense(x.shape[-1])(y)
12
13 class MixerBlock(nn.Module):
14     tokens_mlp_dim: int
15     channels_mlp_dim: int
16     @nn.compact
17     def __call__(self, x):
18         y = nn.LayerNorm()(x)
19         y = jnp.swapaxes(y, 1, 2)
20         y = MlpBlock(self.tokens_mlp_dim, name='token_mixing')(y)
21         y = jnp.swapaxes(y, 1, 2)
22         x = x+y
23         y = nn.LayerNorm()(x)
24         return x+MlpBlock(self.channels_mlp_dim, name='channel_mixing')(y)
25
26 class MlpMixer(nn.Module):
27     num_classes: int
28     num_blocks: int
29     patch_size: int
30     hidden_dim: int
31     tokens_mlp_dim: int
32     channels_mlp_dim: int
33     @nn.compact
34     def __call__(self, x):
35         s = self.patch_size
36         x = nn.Conv(self.hidden_dim, (s,s), strides=(s,s), name='stem')(x)
37         x = einops.rearrange(x, 'n h w c -> n (h w) c')
38         for _ in range(self.num_blocks):
39             x = MixerBlock(self.tokens_mlp_dim, self.channels_mlp_dim)(x)
40         x = nn.LayerNorm(name='pre_head_layer_norm')(x)
41         x = jnp.mean(x, axis=1)
42         return nn.Dense(self.num_classes, name='head',
43                         kernel_init=nn.initializers.zeros)(x)

```

Listing 1: 使用 JAX/Flax 编写的 MLP-Mixer 代码。